



Informe de Cierre y Evaluacion

DGS Rent a Car — Sistema web de renta de autos
Ingeniería de Software · Implementacion, seguridad, pruebas y cierre

Autor	Daniel Alejandro Gonzalez Salazar (AL07076433)
Institucion	Universidad Tecmilenio, Campus Las Torres
Programa	Ingeniería en Desarrollo de Software (IDS)
Repositorio	github.com/danny-svj/DGS-RENT
Metodologia	Scrum — 7 sprints de 2 semanas (07 sep – 09 dic 2026)

1. Resumen del proyecto

DGS Rent a Car es el sistema de renta de autos que he venido diseñando en la materia de Ingeniería de Software, pensado para clientes particulares y para empresas que necesitan un portal B2B. En esta actividad me tocaba pasar de la planificación (arquitectura, diagramas, Avance de proyecto) a tener software realmente funcionando, con seguridad, pruebas automatizadas y un pipeline de integración y entrega continua.

Decidi implementar el modulo de **autenticacion con JWT y roles** (admin/usuario) y el modulo de **vehiculos y reservaciones** como los modulos basicos de esta entrega, y construir alrededor de ellos un producto que se pudiera probar de principio a fin: pagina publica, portal de cliente y panel de administrador, con su respectivo pipeline de CI/CD y escaneo de seguridad automatizado.

2. Comparacion: planificacion vs. ejecucion

La siguiente tabla compara lo que habia planificado en el Avance de proyecto (arquitectura de monolito modular con NestJS, PostgreSQL, Redis, S3 y 7 microservicios documentados) contra lo que realmente implemente en las 9 horas de esta actividad.

Elemento	Planificado	Ejecutado	Diferencia / motivo
Arquitectura	Monolito modular en NestJS, PostgreSQL, Redis, S3, cola de mensajes	API REST en Express.js con SQLite embebido	Simplifique el stack para tener un backend real, desplegable gratis y facil de correr en 9h, sin perder la logica de negocio
Autenticacion	JWT + roles admin/usuario (definido desde el Avance)	Implementado completo: registro, login, bcrypt, JWT (8h), roles admin/user, endpoint /me	Cumplido sin cambios de alcance

Elemento	Planificado	Ejecutado	Diferencia / motivo
Modulo basico requerido	Un modulo basico (ej. registro de personas)	Modulo de vehiculos (CRUD) + reservaciones con reglas de negocio (disponibilidad, calculo de total)	Se amplio el alcance minimo pedido por la actividad
Pruebas unitarias	Cobertura >= 80% con Jest o Pytest	26 pruebas con Jest + Supertest: 97% en auth.js y 96% en vehicles.js	Cumplido, por encima del minimo
CI/CD	Pipeline con GitHub Actions o GitLab para despliegue automatico	4 workflows: tests+cobertura, despliegue a GitHub Pages, escaneo OWASP ZAP, analisis SonarCloud	Cumplido; SonarCloud queda con el pipeline listo pero requiere que yo active un token personal
Seguridad	Escaneo con OWASP ZAP para XSS / SQLi	Escaneo baseline automatico en cada push a main, con reporte descargable como artifact	Cumplido
Calidad de codigo	Analisis con SonarQube (deuda tecnica, code smells)	Workflow de SonarCloud configurado (sonar-project.properties + Action)	Pendiente de activacion: requiere cuenta y token de SonarCloud, fuera del entorno automatizable
Pagos, contratos, notificaciones	Stripe/Mercado Pago, Twilio/SendGrid, firma de contratos (diagrama de microservicios)	No implementados en esta entrega	Fuera del alcance de las 9 horas asignadas; quedan como trabajo futuro (ver seccion 4)
Despliegue	Entorno de prueba accesible	Frontend publico en GitHub Pages + backend con Render Blueprint (render.yaml) listo para un clic	Cumplido; el plan gratuito de Render "duerme" por inactividad y su disco es efimero

3. Lecciones aprendidas

- Simplificar el stack en esta primera iteracion (Express + SQLite en vez de NestJS + PostgreSQL) me permitio entregar software funcionando de verdad en el tiempo disponible, en lugar de quedarme a medias con una arquitectura mas compleja que no iba a alcanzar a desplegar.
- Escribir las pruebas automatizadas desde el inicio me ayudo a encontrar dos errores reales: estaba reutilizando el mismo correo y la misma placa entre pruebas, lo que generaba conflictos de datos que no hubiera notado revisando el codigo a simple vista.
- Configurar los permisos de un Personal Access Token de GitHub no fue trivial: tuve que ajustar por separado los permisos de Contents y de Workflows porque el primer intento de push fue rechazado. La proxima vez lo dejaria documentado desde la planificacion para no perder tiempo en la entrega.
- Separar claramente el "modulo basico" que pedia la actividad (autenticacion + vehiculos) del resto del sistema (pagos, contratos, notificaciones) me ayudo a no perder el foco y entregar esa parte con calidad, en vez de cubrir todo el sistema de forma superficial.
- Dejar la activacion de SonarCloud y de Render como pasos manuales de un clic, en vez de intentar automatizarlos por completo, fue lo mas realista: ambos requieren una cuenta y un token personal que no se pueden generar de forma automatica.

4. Plan de mejora continua

- Migrar de SQLite a PostgreSQL (Render tiene un plan gratuito) para que los datos no se pierdan cada vez que el servicio se reinicia por inactividad.
- Implementar los modulos de pagos y contratos (Stripe o Mercado Pago, firma digital) que ya estan documentados en el diagrama de microservicios pero que no alcance a construir en esta entrega.
- Agregar notificaciones por correo (SendGrid) cuando una reservacion cambia de estado, para que el cliente no tenga que entrar a revisar el portal.
- Ampliar la cobertura de pruebas automatizadas al modulo de reservaciones y de usuarios con archivos de test dedicados (hoy se cubren de forma indirecta dentro de las pruebas de vehiculos).
- Activar SonarCloud con un token real y atender la deuda tecnica y code smells que reporte.
- Explorar un modelo simple de prediccion de demanda por categoria de vehiculo y ciudad, en la misma linea de lo que sugiere la actividad para la prediccion de donaciones.

5. Entregables de esta actividad

Entregable	Donde encontrarlo
Codigo fuente + pipeline CI/CD	github.com/danny-svj/DGS-RENT (carpetas backend/, frontend/, .github/workflows/)
Reporte de pruebas unitarias	Pestana Actions del repositorio -> workflow "CI - Tests y calidad" -> artifact "coverage-report"
Reporte de seguridad (OWASP ZAP)	Pestana Actions -> workflow "Seguridad - OWASP ZAP baseline scan" -> artifact "zap-report"

Analisis SonarCloud	Workflow "Calidad - SonarCloud" (se activa configurando el secreto SONAR_TOKEN, ver README.md)
Software funcionando	Frontend publico: github.io/danny-svj/DGS-RENT · Backend: Render (Blueprint render.yaml)
Este informe (PDF)	docs/DGS-Rent-a-Car_Informe-de-Cierre.pdf en el repositorio

Informe elaborado por Daniel Alejandro Gonzalez Salazar para la actividad de Implementacion, seguridad, pruebas y cierre — materia de Ingeniería de Software, Universidad Tecmilenio.